

# Question Pipeline — Complete Architecture Reference

AI Interview Question Generation Pipeline — every module, every class, every data flow, and the Postgres schema that backs it. Covers both the legacy single-role CLI path (`pipeline_runner.py`) and the production DB-authoritative rollout orchestrator (`orchestrator.py` / `rollout_cli.py`), including the company-research-to-RAG wiring.

Project: `job_readiness_backend_vercel` / `question_pipeline` | Generated: 2026-09-13

---

## Table of Contents

---

- 1. Purpose & Scope of This System
- 2. Two Operating Modes: Legacy CLI vs. Production Orchestrator
- 3. Repository Map
- 4. Configuration Layer — `config.py`
- 5. Domain Models — `models.py`
- 6. Provider Abstraction Layer
- 7. Core Pipeline Services
- 8. Cross-Cutting Infrastructure
- 9. Legacy Single-Role Orchestrator — `pipeline_runner.py`
- 10. Production Rollout Layer
- 11. Database Schema (Migrations 0001 & 0002)
- 12. Company Knowledge → RAG Wiring (Deep Dive)
- 13. End-to-End Data Flow Walkthroughs
- 14. Testing Strategy
- 15. Operational Reference
- 16. Key Invariants & Design Rules (Glossary)

# 1. Purpose & Scope of This System

The question pipeline is a standalone Python package (`question_pipeline/`) inside the FastAPI backend that automatically produces a reusable, quality-gated bank of technical interview questions for a fixed taxonomy of job roles (59 roles in production, a 6-role pilot subset for testing) and, optionally, associates individual questions with real companies/domains as supporting metadata. It does **not** touch any legacy interview table (`interview_questions`, `pre_generated_questions`, `interview_data`, `students`, `student_resumes`, `session_metadata`, `admin_users`, `password_reset_tokens`, `programs`, `program_job_roles`, `university_batch_program`, `session_ratings`, `job_descriptions`) — everything it writes lives in 14 new, purely-additive tables created by two migrations.

The pipeline is a five-stage assembly line for every role (and, separately, for every company profile it maintains): **Research** → **Synthesis** → **RAG Indexing** → **Generation** → **Quality Gate (validation + dedup)** → **Persistence**. Every stage is provider-agnostic (Gemini/OpenAI/Claude/Perplexity/Tavily+Firecrawl/ local/mock are all interchangeable behind small ABCs), every expensive external call is wrapped in rate-limiting and quota/block detection, and every quality decision is fail-closed: a high LLM score can never override a deterministic rejection or a critical LLM-flagged failure.

# 2. Two Operating Modes: Legacy CLI vs. Production Orchestrator

The codebase contains two independent "drivers" over the same underlying services:

| Aspect           | Legacy CLI ( <code>pipeline_runner.py</code> )                                                                                                                                                                                  | Production Orchestrator ( <code>orchestrator.py</code> )                                                                                                                                 |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Entry point      | <code>python -m question_pipeline.pipeline_runner --role "X" --count 20</code>                                                                                                                                                  | <code>python -m question_pipeline.rollout_cli --pilot --batch-size 2</code>                                                                                                              |
| State authority  | Local JSON files under <code>question_pipeline/data/</code> ( <code>state.json</code> , <code>question_bank.json</code> , <code>vector_store.json</code> , <code>validation_attempts.json</code> , <code>cost_log.json</code> ) | PostgreSQL ( <code>knowledge_state</code> , <code>knowledge_version_history</code> , <code>pipeline_jobs</code> , <code>questions</code> and its child tables) — the DB is authoritative |
| Scope            | One role at a time, or the fixed 6-role pilot ( <code>config.PILOT_ROLES</code> )                                                                                                                                               | Any subset of the 59-role production set ( <code>config.PRODUCTION_ROLES</code> ) and/or the 24-company set ( <code>config.COMPANIES</code> ), batched, resumable, idempotent            |
| Checkpointing    | <code>RolePipelineState</code> in <code>state.json</code> , updated per-batch                                                                                                                                                   | <code>pipeline_jobs</code> rows, one per (role, company, job_type), upserted through an explicit state machine                                                                           |
| Change detection | Source-hash compare against <code>state.json</code>                                                                                                                                                                             | Source-hash compare against DB <code>knowledge_state</code> , via <code>freshness.py</code>                                                                                              |
| Typical use      | Local development, one-off testing of a single role, the original 6-role pilot                                                                                                                                                  | The real 59-role/24-company rollout, run in small controlled batches                                                                                                                     |

Both drivers ultimately call the exact same core services (`research_service`, `synthesis_service`, `rag_service`, `generator_service`, `validator_service`) and the exact same `classify_scope()` function — no generation, validation, or scope logic is duplicated between them.

### 3. Repository Map

| Path                                 | Role in the system                                                                                                                        |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <code>config.py</code>               | Single source of truth for every environment-driven setting: providers, models, thresholds, cadences, the 59-role and 24-company lists.   |
| <code>models.py</code>               | All Pydantic v2 data contracts shared across every module.                                                                                |
| <code>providers/</code>              | Provider abstraction layer — one ABC per capability (research/LLM/embedding/validation) plus one concrete class per real or mock backend. |
| <code>research_service.py</code>     | Cadence-aware web research orchestration for a role or a company.                                                                         |
| <code>synthesis_service.py</code>    | Turns raw research text into structured <code>KnowledgeChunks</code> via an LLM.                                                          |
| <code>rag_service.py</code>          | Embeds and cosine-retrieves knowledge chunks (role-level and, now, company-level).                                                        |
| <code>generator_service.py</code>    | Builds the generation prompt (role + RAG + company context) and produces candidate <code>QuestionObjects</code> .                         |
| <code>deduplicator_service.py</code> | 3-tier duplicate detection: exact, normalized/Jaccard, semantic/cosine.                                                                   |
| <code>validator_service.py</code>    | The Phase 2 fail-closed Quality Gate: deterministic checks + a Gemini→OpenAI→Claude LLM rubric ladder.                                    |
| <code>governor.py</code>             | Rate limiting, retry/backoff, and the two pipeline-wide exception types.                                                                  |
| <code>cost_tracker.py</code>         | Per-call USD cost and token accounting.                                                                                                   |
| <code>state_manager.py</code>        | Legacy JSON-file checkpoint store used by <code>pipeline_runner.py</code> .                                                               |
| <code>validation_store.py</code>     | Storage abstraction for validation attempts (in-memory/JSON today, Postgres-ready).                                                       |
| <code>freshness.py</code>            | DB-backed counterpart of <code>state_manager.py</code> , used only by the production orchestrator.                                        |
| <code>pipeline_runner.py</code>      | The legacy single-role/pilot CLI orchestrator.                                                                                            |
| <code>db_loader.py</code>            | Loads local pipeline artifacts (or, for companies, DB-only research) into the migration-0001 schema.                                      |
| <code>orchestrator.py</code>         | The production rollout orchestrator: job checkpointing, change detection, incremental generation, company research.                       |
| <code>rollout_cli.py</code>          | Thin argparse CLI over <code>RolloutOrchestrator</code> .                                                                                 |
| <code>tests/</code>                  | 133 unit/integration tests (see Section 14).                                                                                              |
| <code>docs/</code>                   | This document.                                                                                                                            |
| <code>migrations/0001_*.sql</code>   | 13 new tables: the Knowledge layer + the reusable Question Bank layer.                                                                    |
| <code>migrations/0002_*.sql</code>   | Source URL dedup + the <code>pipeline_jobs</code> checkpoint table (1 new table).                                                         |

`web_research/` (sibling package)

Pre-existing Tavily/Firecrawl HTTP clients reused (not duplicated) by `providers/tavily_firecrawl_provider.py`.

## 4. Configuration Layer — `config.py`

### `is_valid_key(key) → bool`

Module-level helper used by every provider before it trusts a credential. Rejects `None`, non-strings, anything shorter than 8 characters, and anything containing a placeholder substring (`dummy`, `replace_with_real_key`, `placeholder`, `your_`, `todo`, `none`). This is the single check every provider's `_ensure_api_key()/_ensure_configured()` delegates to, so a half-filled-in `.env` value is caught consistently everywhere instead of via ad-hoc `"dummy" in key` checks that some providers used to have.

### `class PipelineConfig` `singleton: config`

A single class whose instance (`config`) is imported everywhere. Every field is read from an environment variable with a sane default, so switching providers/models/thresholds never requires a code change. Notable groups of settings:

- **Paths:** `DATA_DIR` (redirectable via `QUESTION_PIPELINE_DATA_DIR` — this is the test-isolation mechanism), `KNOWLEDGE_DIR`, `STATE_FILE`, `COST_LOG_FILE`, `VECTOR_STORE_FILE`, `QUESTION_BANK_FILE`, `PILOT_REPORT_FILE`.
- **Mode:** `MOCK_MODE` — when true, every provider factory returns a mock/local implementation and no real network call is ever made.
- **Provider selection:** `RESEARCH_PROVIDER`, `LLM_PROVIDER`, `EMBEDDING_PROVIDER`, `VALIDATION_PROVIDER`, `FALLBACK_LLM_PROVIDER`, `ESCALATION_PROVIDER` (+ matching `*_MODEL` fields).
- **API keys:** `GEMINI_API_KEY`, `PERPLEXITY_API_KEY`, `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `TAVILY_API_KEY`, `FIRECRAWL_API_KEY` — never logged.
- **Rate limits:** per-provider RPM caps consumed by `governor.py`.
- **PRICING:** a nested dict of USD-per-million-token input/output cost per provider/model, consumed by `cost_tracker.py`.
- **Dedup thresholds:** `SEMANTIC_SIMILARITY_THRESHOLD` (0.86), `NORMALIZED_OVERLAP_THRESHOLD` (0.85).
- **Question schema constants:** `EXPERIENCE_BANDS`, `PARADIGMS`, `SCOPES`, `PARADIGM_BY_EXPERIENCE_BAND` (single source of truth shared by the generator and the validator so the two can never silently disagree about which paradigm a band expects), `EXPERIENCE_DIFFICULTY_BOUNDS` (a calibration envelope, not a rigid formula — role-specific ceilings are left to the LLM rubric).
- **Quality gate thresholds:** `VALIDATION_PASS_SCORE` (0.70), `VALIDATION_UNCERTAIN_LOW/HIGH` (0.60/0.75 — the escalation band), `DIFFICULTY_DIMENSION_CONSISTENCY_TOLERANCE` (4.0).
- **Cadences:** `ROLE_RESEARCH_CADENCE_DAYS` (30), `COMPANY_RESEARCH_CADENCE_DAYS` (14).
- **Taxonomy:** `PILOT_ROLES` (6), `PRODUCTION_ROLES` (59, exact list), `COMPANIES` (24, exact list) — companies are always contextual metadata, never a primary identity.
- **Company-RAG:** `COMPANY_CONTEXT_MIN_SIMILARITY` (0.35) — the cosine-similarity floor that keeps company context evidence-driven rather than injected (see Section 12).

At import time, `config.DATA_DIR` and `config.KNOWLEDGE_DIR` are created on disk if missing.

## 5. Domain Models — `models.py`

All Pydantic v2 `BaseModel` subclasses. These are the contracts every service passes between each other; none of them do I/O themselves.

### QuestionObject

The canonical shape of one interview question, end to end — from a raw LLM candidate through validation to a persisted DB row. Fields: `question`, `role`, `experience_band`, an overall `difficulty` (1–10) plus the **7 difficulty dimensions** (`technical_depth`, `problem_complexity`, `architecture_complexity`, `troubleshooting`, `business_complexity`, `decision_making`, `leadership_ownership`, each 1–10), `question_type`, `paradigm` (one of the 5 paradigms), `mandatory_skills`, `scope` (UNIVERSAL/DOMAIN/COMPANY), `domains`, `applicable_companies`, `source_references` (provenance strings, including RAG chunk citations), `knowledge_version`, `generated_at`.

### ResearchOutput

The result of one research call, for a role or a company (when `company` is set, `role` holds the company name too — the convention every company-only artifact in this system follows). Carries `technologies`, `engineering_practices`, `responsibilities`, `interview_topics`, `trends`, the full `raw_summary` text, `source_references`, a `source_hash` (sha256 of the raw text — the change-detection key everywhere in the system), `knowledge_version`, and `actual_provider` (recorded explicitly so it's always knowable which provider genuinely produced this data, never assumed from config).

### KnowledgeChunk

One embeddable unit of synthesized knowledge. `role: str` is required (non-Optional) and `company: Optional[str]` defaults to `None`. For a standalone company knowledge chunk, `role` holds the company name (since neither the Pydantic model nor the DB column allow a null role) and `company` is the real discriminator used everywhere retrieval needs to tell role-chunks and company-chunks apart. Also carries `topic`, `text`, `embedding`, `embedding_model`, `source_reference`, `knowledge_version`.

### VectorStoreData

The on-disk container for `vector_store.json`: `embedding_provider`, `embedding_model`, and the full `chunks: List[KnowledgeChunk]`. Versioned by provider+model so `RAGService` can detect an incompatible embedding space and invalidate the whole store rather than mixing vectors from two models.

### ValidationResult

A simple pass/fail/score/tier record; superseded in the actual pipeline by the richer `QualityGateResult` below but kept as part of the abstract `ValidationProvider` interface.

### QualityGateResult

The structured, auditable output of the Phase 2 quality gate. Fields: `approved`, `quality_score`, `critical_failure`, `rejection_reasons` (list of machine-readable reason codes), `check_results` (the LLM's own per-dimension true/false checks), `validation_tier` (`primary_gemini`/`fallback_gpt`/`escalation_claude`), `validation_provider`, `validation_model`, `reasoning`, `uncertain`.

**Fail-closed contract:** `approved` must never be `True` when `critical_failure` is `True`. Enforced in `ValidatorService._merge()` in code, and redundantly at the database level by `chk_qva_fail_closed` once migration 0001 is applied.

Method `to_attempt_record(role, candidate_question_text, question_id)` converts this in-process result into the exact row shape `question_validation_attempts` expects.

### ValidationAttemptRecord

An exact mirror of one `question_validation_attempts` row — one instance per validation attempt, approved or rejected. `id` is `None` until a store assigns one.

### CostRecord

One logged API call: `provider`, `model`, `task`, `input_tokens`, `output_tokens`, `estimated_cost`, `timestamp`, `status`.

### RolePipelineState

The legacy JSON-file checkpoint record for one role or company (used by `state_manager.py`): `status` (PENDING/RUNNING/COMPLETED/WAITING\_FOR\_QUOTA/BLOCKED/FAILED/RETRY/SKIPPED), `current_batch/total_batches`, `target_questions`, `accepted_questions_count`, cadence timestamps, `knowledge_version`, `source_hash`, `reason`.

### PilotQualityReport

The comprehensive end-of-pilot audit object `pipeline_runner.run_pilot()` produces: totals (generated/accepted/rejected/duplicates/validation\_failures), distributions (scope/difficulty/experience), per-role `role_correctness`, full API usage and cost breakdowns by provider and by task, and any errors encountered. Saved to `pilot_quality_report.json`.

## 6. Provider Abstraction Layer

Every external dependency (web research, LLM generation, embeddings) is accessed only through a small ABC. Concrete providers are chosen purely by config/env vars via three factory functions, so swapping Gemini for OpenAI, or a real provider for its mock, never touches calling code.

### 6.1 providers/base.py — the contracts

**extract\_json\_object(text) → Any** (module function, shared by every LLM provider)  
Tries, in order: (1) parse as-is; (2) strip a leading/trailing ```json code fence and parse; (3) `json.JSONDecoder().raw_decode()` to parse only the *first* complete JSON value and discard trailing content. Step 3 is what fixed a real production bug ("Extra data: line 81 column 1") where Gemini appended stray content after a complete, valid JSON object — plain `json.loads()` refuses to parse when trailing content exists, silently losing an entire generation batch. This function never invents or repairs field content, only extracts what is verbatim present.

**LLMResult**  
Plain wrapper for one LLM call's output: `text`, `parsed data`, `input_tokens`, `output_tokens`, `model`, `provider`.

**ResearchProvider (ABC):** `provider_name`, `research_role(role)`, `research_company(company)` — both return a `ResearchOutput`.  
**LLMProvider (ABC):** `provider_name`, `generate_json(prompt, system_prompt, task_name)`, `generate_text(...)`.  
**EmbeddingProvider (ABC):** `provider_name`, `embed_texts(texts, task_name)`, `embed_query(text, task_name)`.  
**ValidationProvider (ABC):** `validate_question(question_data, role_context)` — declared for completeness; the real pipeline's validation ladder is implemented directly in `ValidatorService`, not through this interface.

### 6.2 providers/factory.py — selection logic

Three functions, each following the same pattern: if `config.MOCK_MODE`, always return the mock/local implementation (guaranteeing zero real API calls in tests); otherwise dispatch on the relevant config string.

| Function                                              | Real-mode dispatch                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>get_research_provider()</code>                  | <code>perplexity</code> → <code>PerplexityResearchProvider</code> ;<br><code>tavily_firecrawl</code> → <code>TavilyFirecrawlResearchProvider</code> ; <code>gemini</code> → explicitly raises <code>ProviderBlockedException</code> (not yet enabled); anything else → blocked with a clear message.                                                                                   |
| <code>get_llm_provider(provider_override=None)</code> | <code>gemini</code> → <code>GeminiLLMProvider</code> ,<br><code>openai</code> → <code>OpenAILLMProvider</code> ,<br><code>anthropic</code> → <code>ClaudeLLMProvider</code> , else defaults to Gemini. <code>provider_override</code> is how the validator's fallback/escalation tiers request a <i>specific</i> provider regardless of the primary <code>LLM_PROVIDER</code> setting. |
| <code>get_embedding_provider()</code>                 | <code>gemini</code> → <code>GeminiEmbeddingProvider</code> ,<br><code>local</code> → <code>LocalEmbeddingProvider</code> , else blocked.                                                                                                                                                                                                                                               |

`get_validation_provider()`

Thin alias for  
`get_llm_provider(config.VALIDATION_PROVIDER)`.

### 6.3 Gemini — `providers/gemini_provider.py`

#### **GeminiLLMProvider**

Primary generation/synthesis/validation LLM. `generate_json()` configures the SDK once, calls `genai.GenerativeModel(...).generate_content()` off the event loop via `loop.run_in_executor`, requests `response_mime_type="application/json"`, extracts JSON via `extract_json_object()`, estimates or reads real token usage, and logs cost via `cost_tracker.record_usage()`. Every call is routed through `governor.execute_with_retry()` for throttling/backoff. `generate_text()` is the same shape without the JSON response format.

#### **GeminiEmbeddingProvider**

Configurable-model embedding provider. On first use it calls `_verify_model()`, which lists the account's actually-available embedding models and silently substitutes the configured model for the first available one if the configured name isn't currently active — a defensive measure against a dead/renamed model breaking the whole pipeline. Batches `embed_texts()` in chunks of 20. Both methods log usage through `cost_tracker` and go through the governor.

### 6.4 OpenAI — `providers/openai_provider.py`

#### **OpenAILLMProvider**

Tier-2 fallback validation provider (model/provider fully configurable via `FALLBACK_LLM_PROVIDER/FALLBACK_LLM_MODEL`). Calls `POST /v1/chat/completions` with `response_format={"type":"json_object"}` via `httpx.AsyncClient`, mapping HTTP 429→`QuotaExhaustedException` and 401/403→`ProviderBlockedException` before the governor's generic retry logic even runs.

### 6.5 Claude — `providers/claude_provider.py`

#### **ClaudeLLMProvider**

Tier-3 escalation validation provider (`ESCALATION_PROVIDER/ESCALATION_MODEL`). Calls Anthropic's Messages API directly (`x-api-key` header, `anthropic-version: 2023-06-01`), concatenates all `text` content blocks from the response, then runs the result through the same `extract_json_object()` used everywhere else.

### 6.6 Perplexity — `providers/perplexity_provider.py`

#### **PerplexityResearchProvider**

The original research provider: a single Sonar chat-completion call per role/company, asked directly for a structured interview-intelligence dossier. Returns Perplexity's own `citations` as `source_references`. Still fully supported and selectable via `RESEARCH_PROVIDER=perplexity`, but the pipeline has since moved to Tavily+Firecrawl as its primary real research stack.

### 6.7 Tavily + Firecrawl — `providers/tavily_firecrawl_provider.py`

#### **TavilyFirecrawlResearchProvider**

The current real research stack: **discovery** via Tavily search, then **extraction** via Firecrawl scrape, built on top of the pre-existing `web_research.clients.TavilySearchClient / FirecrawlScrapeClient` (reused, not reimplemented). Internal flow (`_research()`):

1. Tavily search for up to 5 candidate sources; raises if none are found — never fabricates a dossier from nothing.
2. Rank by Tavily's own `relevance_score`, keep the top 3.
3. Firecrawl-scrape those 3 URLs.
4. Build the final source set: fully-Firecrawl-extracted sources first; sources Firecrawl couldn't reach but that still have a Tavily snippet are kept as an explicit, honest degraded fallback (never fabricated text); if every source fails extraction and has no snippet either, raises rather than inventing content.
5. `raw_summary` is assembled verbatim from the retrieved source text; `source_hash` is sha256 of that summary — this is the value every change-detection check in the system compares.

`research_role(role)` and `research_company(company)` both call the same `_research()` with a role- or company-flavored query string; the company variant stamps `company=company` onto the resulting `ResearchOutput` via `model_copy(update=...)`.

## 6.8 Local embeddings — `providers/local_provider.py`

### **LocalEmbeddingProvider**

Deterministic, zero-cost, zero-network embedding used whenever `MOCK_MODE=true` or `EMBEDDING_PROVIDER=local`. `_text_to_vector()` hashes each token (and each 3-gram of each token) into one of `dim` (default 128) buckets with a sign derived from the hash, then L2-normalizes the result — a crude but stable and reproducible bag-of-hashed-features embedding, good enough for exercising RAG/dedup logic in tests without a real embedding call.

## 6.9 Mocks — `providers/mock_provider.py`

### **MockResearchProvider**

Returns a fixed, realistic-looking `ResearchOutput` instantly, for both roles and companies — no network call.

### **MockLLMProvider**

The workhorse of the whole test suite. Branches on `task_name`: for `"synthesis"` returns 4 canned knowledge modules; for `"validation"` returns a fixed high-passing score; otherwise it parses `role/experience_band/paradigm/target_scope` straight out of the prompt text via regex and returns a full set of hand-curated, realistic candidate questions from `ROLE_BAND_QUESTIONS` (a large literal dictionary of 20 pre-written, role- and band-specific questions per pilot role) so mock-mode integration tests still exercise realistic-looking difficulty/skill/scope data instead of placeholder text.

**MockEmbeddingProvider** = `LocalEmbeddingProvider` (direct alias, not a subclass).

## 7. Core Pipeline Services

### 7.1 `research_service.py`

**ResearchService** `singleton: research_service`

**execute\_research(role, force=False)** — the role-level entry point used by the legacy CLI path. Checks `state_manager.is_role_due_for_research()` (skips on cadence unless forced), calls `provider.research_role(role)`, catches `QuotaExhaustedException/ProviderBlockedException` and records them into `state_manager` before re-raising, compares the new `source_hash` against the stored one (skip if unchanged), then writes the research JSON to `knowledge_base/{slug}_research.json` and updates `state_manager` with the new cadence window.

**execute\_company\_research(company)** — the company-level counterpart used by the production orchestrator. Cadence/hash gating for companies is entirely the orchestrator's responsibility (via `freshness.py`, DB-authoritative) done *before* this is called, so this method always executes unconditionally when invoked — it calls `provider.research_company(company)` and writes `knowledge_base/company_{slug}_research.json`.

### 7.2 `synthesis_service.py`

**SynthesisService** `singleton: synthesis_service`

**synthesize(research: ResearchOutput) → List[KnowledgeChunk]** asks the configured LLM to compress a research dossier into 4–6 distinct "knowledge modules" (topic + content), falling back to a fixed 3-chunk split of the raw summary if the LLM returns nothing usable. Each resulting `KnowledgeChunk` is stamped `role=research.role` — which, critically, is exactly the company name for a company `ResearchOutput`, since nothing here needs to know or care whether the input came from a role or a company. Persists to `knowledge_base/{slug}_synthesized.json`.

### 7.3 `rag_service.py`

**RAGService** `singleton: rag_service`

Owns the in-memory + on-disk (`vector_store.json`) list of every indexed `KnowledgeChunk`, and all cosine-similarity retrieval.

- **`_load()/save()`**: on load, if the stored `embedding_provider/embedding_model` differ from the currently-configured ones, the entire store is invalidated (cleared) rather than mixing incompatible embedding spaces.
- **`index_chunks(chunks)`**: embeds all chunk texts in one batch call, replaces any chunk sharing an existing `chunk_id`, appends the rest, persists.
- **`clear_role_chunks(role)`**: used by `--force` reruns to wipe only one role's chunks.
- **`_cosine_rank(query_arr, pool) / _embed_normalized_query(query)`**: the shared, L2-normalized cosine-ranking core used identically by both retrieval methods below, so they can never drift out of sync.
- **`retrieve_context(role, experience_band, paradigm, top_k=3)`**: role-only retrieval. Filters to chunks with `company is None` and a matching `role` (falling back to any role-chunk if none match this exact role); this `company is None` guard is explicit defense-in-depth so a company chunk can never leak into role-only retrieval even though it technically shares the `role` column's storage slot.
- **`retrieve_company_context(role, experience_band, paradigm, companies=None, top_k=1)`** — added for the company-RAG feature; see Section 12 for the full design rationale.

### 7.4 `generator_service.py`

**GeneratorService** `singleton: generator_service`

`_generate_json_with_bounded_retry(prompt, role, experience_band)`: calls the LLM, and on exactly one `json.JSONDecodeError` retries the identical prompt once more (never repairs or edits content) — this is the second layer of defense (after `extract_json_object`) against the "Extra data" malformed-JSON failure mode observed in a real pilot; `QuotaExhaustedException/ProviderBlockedException` propagate immediately, never retried here.

`generate_batch_for_band(role, experience_band, count, target_scope, domain=None)` is the main entry point. It: resolves the `paradigm` for this band from `config.PARADIGM_BY_EXPERIENCE_BAND`; retrieves up to 2 role-context chunks and up to 1 company-context chunk via `rag_service`; assembles a detailed prompt containing the role invariance rule, target parameters, the merged RAG context (with a `[Company Context -- ...]` block appended only when one was actually retrieved), explicit scope-classification rules for the LLM to follow, and calibration guidance per experience band (with concrete AVOID/PREFER lists for the 0–1yr band to stop trivial/definition-only questions); calls the bounded-retry generator; force-overwrites `role/experience_band/paradigm/source_references/knowledge_version/generated_at` on every returned item (role invariance is never left to the LLM's discretion) before constructing each `QuestionObject`.

## 7.5 deduplicator\_service.py

**DeduplicatorService** `singleton: deduplicator_service`

3-tier duplicate detection, cheapest check first:

1. **Exact match**: case/whitespace-normalized string equality.
2. **Normalized overlap**: `token_jaccard_similarity()` over stopword-stripped token sets, rejecting at  $\geq \text{NORMALIZED\_OVERLAP\_THRESHOLD}$  (0.85).
3. **Semantic cosine similarity**: embeds the candidate and every bank item (caching bank embeddings in `_cached_embeddings` across calls) and rejects at  $\geq \text{SEMANTIC\_SIMILARITY\_THRESHOLD}$  (0.86). On an embedding-call exception, logs a warning and falls back to the already-completed normalized check rather than crashing.

Also exports `normalize_text()` and `token_jaccard_similarity()` as standalone helpers.

## 7.6 validator\_service.py — the Phase 2 Quality Gate

The single most important correctness component in the system. Every candidate question passes through here before it can reach the approved bank.

**ValidatorService** `singleton: validator_service`

### Deterministic checks (no LLM, cannot be argued with)

| Method                              | Rejects when...                                                                       |
|-------------------------------------|---------------------------------------------------------------------------------------|
| <code>_check_role_invariance</code> | <code>q.role</code> doesn't exactly match the expected role                           |
| <code>_check_difficulty_band</code> | <code>difficulty</code> falls outside <code>EXPERIENCE_DIFFICULTY_BOUNDS[band]</code> |
| <code>_check_paradigm</code>        | <code>paradigm</code> doesn't match <code>PARADIGM_BY_EXPERIENCE_BAND[band]</code>    |
| <code>_check_scope</code>           | <code>scope</code> isn't one of UNIVERSAL/DOMAIN/COMPANY                              |

|                                            |                                                                                                             |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <code>_check_dimension_consistency</code>  | overall <code>difficulty</code> diverges from the mean of the 7 dimensions by more than the tolerance (4.0) |
| <code>_check_duplicate</code> (async)      | <code>deduplicator_service.check_duplicate()</code> flags a match against the existing bank                 |
| <code>_retrieve_rag_context</code> (async) | zero RAG chunks are retrievable at all → <code>insufficient_rag_grounding</code>                            |

### LLM rubric ladder (Gemini → OpenAI → Claude)

`_build_prompt()` hands the LLM the candidate's full data plus the retrieved RAG context and asks it to score 12 explicit dimensions (`role_correctness`, `technical_correctness`, `experience_seniority_fit`, `difficulty_fit`, `clarity`, `real_interview_relevance`, `reasoning_depth`, `no_hallucination`, `grounded_in_research`, `not_trivial`, `correct_paradigm`, `correct_dimensions`), returning a score, a pass/fail, an `uncertain` flag, and a list of `critical_failures` reason codes. `_run_llm_tier()` executes one tier and normalizes its output (treating any exception as a fail-closed rejection, never a guess). `_merge()` is where the fail-closed contract is enforced in code: `approved = (not critical_failure) and passed and not uncertain` — a critical failure (deterministic or LLM-flagged) makes approval impossible regardless of score.

`validate_question(candidate, existing_bank, expected_role)` is the public entry point:

1. Run all deterministic checks (once) and retrieve RAG context.
2. Always run Tier 1 (Gemini) — even when a deterministic check already failed, so the audit trail always has an LLM opinion. If deterministic checks failed, return immediately (nothing for a second opinion to resolve).
3. If Tier 1 is confident (not `uncertain`), return its result.
4. Otherwise escalate to Tier 2 (fallback provider); if still uncertain, escalate to Tier 3 (escalation provider).
5. If Tier 3 is *still* uncertain, the result is force-rejected with `unresolved_uncertainty_after_full_ladder` — an unresolved "maybe" is never approved.
6. If Tier 2/3 are unreachable (missing credentials, quota exhausted), Tier 1's own uncertain result is *not* trusted as a fallback approval — it is force-rejected with `escalation_unavailable_uncertain_result`. This is a deliberate strengthening: an unconfirmed shaky score is a rejection, never an approval.

## 8. Cross-Cutting Infrastructure

### 8.1 `governor.py`

#### **QuotaExhaustedException / ProviderBlockedException**

The two exception types every provider and service uses to signal, respectively, "the provider's quota/rate limit is exhausted" and "the provider cannot run at all (bad/missing credentials, forbidden)". Both carry `provider`, `model`, `message`. Every layer of the pipeline treats these as first-class control flow — never silently swallowed, always propagated up to whatever checkpointing layer (`state_manager` or `pipeline_jobs`) needs to pause safely.

#### **RateLimitGovernor** `singleton: governor`

`throttle(provider)` sleeps just enough to respect that provider's configured RPM before a call.

`execute_with_retry(provider, model, func, ..., max_retries=3)` wraps a call with jittered exponential backoff, inspecting the exception message to distinguish a transient 429/quota condition (backs off, then raises `QuotaExhaustedException` after the final retry) from an auth/permission failure (raises `ProviderBlockedException` immediately, no retry) from a generic transient error (retried with a smaller fixed backoff).

### 8.2 `cost_tracker.py`

#### **CostTracker** `singleton (via __new__): cost_tracker`

`record_usage(provider, model, task, input_tokens, output_tokens, status)` looks up per-million-token pricing from `config.PRICING`, computes an exact cost, appends a `CostRecord`, and persists the whole list to `cost_log.json` after every single call (so a crash mid-run never loses cost history). `get_summary()` aggregates total cost, cost-by-provider, cost-by-task, and token/call counts by provider — this is what `PilotQualityReport.estimated_api_cost` is built from.

### 8.3 `state_manager.py`

#### **StateManager** `singleton: state_manager`

The legacy JSON-file (`state.json`) checkpoint store keyed by role/company name, holding one `RolePipelineState` per key. `get_role_state()/update_role_state()` (partial field updates, always re-saves), `mark_waiting_for_quota()/mark_blocked()` (thin wrappers over `update_role_state`), `reset_role_state()` (used by `--force`), `is_role_due_for_research()` (cadence check), `is_hash_unchanged()` (change detection), `list_all_states()` (used by `--status`). This class is untouched by the production orchestrator work — it keeps serving the legacy single-role CLI path exactly as before.

### 8.4 `validation_store.py`

**FailClosedViolation(ValueError)** — raised by `_assert_fail_closed_invariants()` when a caller tries to record an attempt that would violate the fail-closed contract in code, mirroring the DB's own `chk_qva_*` CHECK constraints as defense-in-depth even before the schema exists.

**ValidationAttemptStore (ABC):** `record()`, `list_for_role()`, `list_for_question()`.

**InMemoryValidationAttemptStore:** the store actually used by `pipeline_runner.py` today. Appends immediately (no batching) and, if given a `persist_path`, rewrites the whole `validation_attempts.json` file after every `record()` call — this per-candidate durability is what makes mid-batch quota-exhaustion checkpointing meaningful (everything recorded before the failure is already durable).

**PostgresValidationAttemptStore**: a fully-implemented, ready-to-swap-in real implementation against `question_validation_attempts`, deliberately **not** the default anywhere today and not exercised by any test — wiring it in when desired is a one-line change (`QuestionPipelineRunner.validation_store = PostgresValidationAttemptStore(...)`). It never pre-validates beyond what the DB itself enforces; a raised `CheckViolation` on an invalid row is the intended failure mode, not a bug to hide.

## 8.5 `freshness.py`

**bump\_version(current)** → **str** (module function) — `v1.0`→`v1.1`, `v1.9`→`v1.10`, `None/unparseable`→`v1.0/v1.1`.

### **KnowledgeFreshnessTracker**

The DB-backed mirror of `state_manager.py`'s cadence/hash logic, used exclusively by the production orchestrator, reading/writing `knowledge_state` (current pointer, updated in place) and `knowledge_version_history` (append-only ledger) instead of a JSON file. `get_state(role, company=None)`, `is_due(role, company=None)` (role cadence 30d, company cadence 14d), `is_hash_unchanged()`, `record_no_change()` (refreshes the freshness timestamp without bumping the version or touching the question bank), `record_change()` (bumps `current_knowledge_version` in `knowledge_state` and appends an immutable row to `knowledge_version_history`, so every prior knowledge version stays inspectable forever), and `mark_status()` (mirrors `mark_waiting_for_quota/mark_blocked`, DB-side).

## 9. Legacy Single-Role Orchestrator — `pipeline_runner.py`

### **QuestionPipelineRunner** `singleton: pipeline_runner`

Loads/saves `question_bank.json` directly (`_load_question_bank/_save_question_bank`), and owns an `InMemoryValidationAttemptStore` by default.

- `clear_role_questions(role)` / RAG's `clear_role_chunks(role)`: the surgical, role-scoped reset used by `--force`.
- `print_diagnostics(role, force)`: prints mode/provider/model/fallback-and-escalation-tier readiness at the start of every run, without ever printing a secret value.
- `validate_prerequisites(role)`: in real (non-mock) mode, checks that every credential the configured research/LLM/embedding providers actually need is present and non-placeholder *before* any paid call is made; on any gap, marks the role BLOCKED and raises immediately.
- `classify_scope(candidate, target_scope)`: the same function documented in Section 12 below — shared verbatim with the production orchestrator via direct import, never duplicated.
- `run_role_pipeline(role, target_questions=20, force=False)`: the full single-role pipeline. Skips immediately if already COMPLETED with enough questions (unless forced). On `force`, resets state + bank + RAG chunks for that role only. Runs Research→Synthesis→RAG-indexing only when starting from batch 0 (a resume skips straight to generation). For each of the 5 experience bands (in order, with the batch index deciding `target_scope`: band 1→DOMAIN, band 4→COMPANY, else UNIVERSAL), generates a batch, then for every candidate: classifies scope, runs it through the quality gate, and immediately records the attempt (approved or rejected) — this per-candidate durability is the checkpointing guarantee: a `QuotaExhaustedException` on candidate N+1 leaves 1..N already safely persisted. Checkpoints `current_batch/accepted_questions_count` after every batch. Final status is COMPLETED if ≥70% of the target was reached, else PARTIAL.
- `run_pilot(force=False)`: runs `run_role_pipeline` for each of the 6 `PILOT_ROLES` in sequence, halting safely (not erroring) the moment any role reports WAITING\_FOR\_QUOTA/BLOCKED, then builds and saves a full `PilotQualityReport` (`_build_pilot_report/_save_pilot_report`) from whatever was accepted.

CLI (`__main__`): `--pilot, --role NAME --count N, --force, --resume` (identical to `--pilot` today — per-role checkpointing already makes resume automatic), `--status` (prints `state_manager.list_all_states()` + `cost_tracker.get_summary()`).

## 10. Production Rollout Layer

### 10.1 `db_loader.py`

**LoadReport** (dataclass) — every counter one call to the loader can produce: inserted/already-present counts for questions, question\_companies/domains/skills/sources/embeddings, validation\_attempts, research\_documents/chunks/sources, plus `inserted_question_ids` and `errors`.

### **QuestionBankLoader**

Loads local pipeline artifacts (or, for a company, DB-only research artifacts) into the 13 migration-0001 tables. Writes *only* to those tables — contains no SQL referencing any legacy table. One Postgres transaction per call: any failure rolls back the entire load.

- `load_role(role, question_bank, validation_attempts, since=None)` (async): computes real embeddings for every candidate-new question *before* opening the DB transaction (no network I/O inside a transaction), then inside one transaction: loads research/knowledge via `_load_research_and_knowledge()`, inserts each question

via `_load_question()` (`ON CONFLICT (role, question_text_normalized) DO NOTHING` — idempotent by construction), and for newly-inserted questions only, loads relations (`_load_question_relations`) and the embedding (`_load_question_embedding`); finally loads validation attempts scoped to `since` so a repeated load never double-logs the same run's attempts.

- `load_company_research(company)` (sync): the company-only counterpart added for the company-RAG feature (Section 12) — loads only research/knowledge (no questions/embeddings/validation attempts) for a standalone company profile.
- `_load_question(cur, q, report)`: the idempotent question insert with a `SELECT` fallback to recover the existing row's id on conflict.
- `_load_question_relations(cur, qid, q, chunk_id_to_pk, report)`: inserts `question_companies/question_domains/question_skills` rows, and `question_sources` provenance rows — parsing a "RAG Chunk: <id> | ..." reference back to a real `research_chunks.id` (falling back to a direct DB lookup if that chunk wasn't loaded earlier in *this specific call*, so provenance still resolves correctly across separate incremental loads) or, failing that, storing the raw reference string.
- `_load_question_embedding(cur, qid, emb, report)`: `ON CONFLICT (question_id, embedding_model) DO NOTHING`.
- `_load_validation_attempts(...)`: logs every attempt (approved and rejected), linking `question_id` only for approved ones whose normalized text matches an inserted question.
- `_load_vector_store_chunk_embeddings(role=None, company=None)`: reads `vector_store.json` filtered by role or by company (mutually exclusive), for attaching real embedding vectors to `research_chunks` rows.
- `_load_research_and_knowledge(cur, role, report, company=None)`: the shared core for both `load_role()` and `load_company_research()`. When `company` is set, reads `company_{slug}_research.json/{slug}_synthesized.json` instead of the role-named files, and stores each resulting `research_chunks` row with `role=company` (the NOT-NULL-safe slot) and `company=company` (the real discriminator) — see Section 12. Also upserts `research_sources` idempotently via `ON CONFLICT (url_normalized) DO NOTHING` (migration 0002), and, for the role-only path, syncs `knowledge_state` from the legacy `state.json`.

Module-level helpers: `slugify()`, `normalize_question_text()` (mirrors the DB's own generated column exactly), `vector_literal()` (formats a Python float list as a `pgvector`/halfvec literal string).

## 10.2 orchestrator.py

Introduces **no new** research/synthesis/generation/validation/dedup logic — it purely coordinates the existing services against DB-authoritative state instead of local files.

**UnitResult** (dataclass) — the uniform return shape of every orchestrator operation: `role`, `company`, `job_type`, `status`, `reason`, `metrics`.

### PipelineJobStore

Thin read/write wrapper around `pipeline_jobs` (migration 0002). `upsert(role, company, job_type, status, reason, metrics, mark_started, mark_completed)` is a single idempotent `INSERT ... ON CONFLICT (role, COALESCE(company, ''), job_type) DO UPDATE` — always safe to call repeatedly for the same logical unit as it progresses through its state machine. `get()`, `list_all(roles=None)` (used by `--status`).

**synthesize\_and\_index\_company\_research(company, research\_output)** (module function, async) — synthesizes a company `ResearchOutput` (reusing `synthesis_service.synthesize()` verbatim) and stamps `chunk.company = company` explicitly on every resulting chunk before indexing into `rag_service`. See Section 12.

`fetch_existing_bank_from_db(db_config, role)` (module function) — reconstructs a dedup-usable `List[QuestionObject]` from the DB's *current* approved bank for a role. This is what makes semantic dedup during a refresh DB-authoritative rather than dependent on local file/process memory.

### RolloutOrchestrator

Owns a `PipelineJobStore`, a `KnowledgeFreshnessTracker`, and a `QuestionBankLoader`.

- `get_pending_roles(roles)`: a role is pending unless its last `ROLE_GENERATION` job was `COMPLETED/SKIPPED_NO_CHANGE` and it isn't due again per `freshness.is_due()`.
- `process_role(role, dry_run=False)`: the main per-role state machine (see the full walkthrough in Section 13.2). Every transition is checkpointed to `pipeline_jobs` before/after the work it describes, so a crash mid-role leaves an inspectable, resumable status. On unchanged research content, short-circuits to `SKIPPED_NO_CHANGE` without touching the question bank. On changed content: bumps the knowledge version, synthesizes + indexes RAG, then generates a **full pilot-sized batch on a role's first run** or a **small top-up batch** (`REFRESH_TOP_UP_QUESTIONS_PER_ROLE`, default 8) on a refresh, validating and deduping every candidate against the DB's current bank (via `fetch_existing_bank_from_db` plus whatever was already approved earlier in this same run), then loads only the newly-approved questions. Quota/blocked mid-generation triggers `_load_partial()` so whatever was already approved before the interruption is never lost.
- `process_company_research(company, dry_run=False)`: the company-level state machine; see Section 12.4 for the full walkthrough.
- `run_batch(roles, batch_size, dry_run=False)`: processes up to `batch_size` pending roles, stopping the batch (not erroring) the moment any role reports `WAITING_FOR_QUOTA/BLOCKED` — every other pending role stays untouched and safe to resume in a later invocation.

## 10.3 rollout\_cli.py

Thin argparse CLI over `RolloutOrchestrator`. Key flags: `--pilot` (6 roles), `--roles "A,B"` (explicit, validated against the 59-role set), `--all-roles` (must be explicit, never implied), `--companies "A,B" / --all-companies` (validated against the 24-company set — the only entry point that actually triggers `process_company_research()`), `--batch-size N` (applies independently to roles and companies), `--dry-run` (zero API calls, zero DB writes), `--status` (prints current `pipeline_jobs` rows and exits). Refuses to run a non-dry-run batch under `MOCK_MODE=true`, and refuses if the real research/LLM keys aren't valid. Roles and companies can be combined in one invocation, each processed as its own independent job type.

## 11. Database Schema (Migrations 0001 & 0002)

Both migrations are purely additive, wrapped in a single transaction, and every `CREATE` is `IF NOT EXISTS` (safe to re-run). Neither references any of the 13 pre-existing legacy application tables. Embeddings use `halfvec(3072)` (not the plain `vector` type, which caps indexable dimensions at 2000) with HNSW cosine indexes, matching the verified 3072-dim output of `gemini-embedding-2`.

### 11.1 Migration 0001 — Knowledge Layer + Reusable Question Bank (13 tables)

| Table                                     | Purpose & key constraints                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>research_sources</code>             | One row per web source (URL). Separate <code>discovery_provider/extraction_provider</code> columns since Tavily (discovery) and Firecrawl (extraction) are frequently different services.                                                                                                                                                                                                                                                                  |
| <code>research_documents</code>           | One row per research run for a role or company (maps to <code>ResearchOutput</code> ). Insert-only by convention — a refresh creates a new row.                                                                                                                                                                                                                                                                                                            |
| <code>research_document_sources</code>    | Many-to-many join: which sources fed which document.                                                                                                                                                                                                                                                                                                                                                                                                       |
| <code>research_chunks</code>              | Embedded knowledge chunks (replaces <code>vector_store.json</code> as the DB-authoritative form). <code>role TEXT NOT NULL</code> , <code>company TEXT</code> nullable, <code>embedding halfvec(3072)</code> , HNSW index. <code>CHECK</code> ensures embedding/model/provider are filled in together or not at all.                                                                                                                                       |
| <code>knowledge_state</code>              | Current cadence/status pointer per (role, company) — the DB analogue of <code>state.json</code> . Unique on <code>(role, COALESCE(company, ''))</code> , updated in place.                                                                                                                                                                                                                                                                                 |
| <code>knowledge_version_history</code>    | <b>Append-only</b> ledger of every <code>knowledge_state</code> transition — never updated/deleted, so prior versions are always inspectable.                                                                                                                                                                                                                                                                                                              |
| <code>questions</code>                    | Canonical reusable questions. One row per question regardless of how many companies/domains reference it. <code>question_text_normalized</code> is a generated column; <code>UNIQUE(role, question_text_normalized)</code> . 7 difficulty dimensions + overall <code>difficulty</code> , each <code>CHECK BETWEEN 1 AND 10</code> . <code>paradigm/scope/experience_band</code> are all <code>CHECK</code> -constrained to their exact allowed value sets. |
| <code>question_companies</code>           | Many-to-many: which companies a question applies to. <code>company_name</code> is free text (no canonical companies master table exists anywhere in this database).                                                                                                                                                                                                                                                                                        |
| <code>question_domains</code>             | Many-to-many: business/technology domains.                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <code>question_skills</code>              | Many-to-many: mandatory/associated skills.                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <code>question_sources</code>             | Provenance: links a question to the research chunk/document/raw citation that justifies it. Partial unique indexes prevent double-linking the same chunk/document/reference to the same question.                                                                                                                                                                                                                                                          |
| <code>question_embeddings</code>          | Semantic-dedup embeddings per question, keyed by <code>(question_id, embedding_model)</code> so re-embedding with a new model never destroys history.                                                                                                                                                                                                                                                                                                      |
| <code>question_validation_attempts</code> | <b>Append-only</b> audit log of every validation attempt, approved and rejected. <code>chk_qva_fail_closed CHECK (NOT (critical_failure AND approved))</code> and <code>chk_qva_approved_has_question CHECK (NOT (approved AND question_id IS NULL))</code> enforce the fail-closed contract at the database level — it is physically impossible to store a row that violates it.                                                                          |

### 11.2 Migration 0002 — Source Dedup + Orchestration Checkpointing

| Change                                                                            | Purpose                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>research_sources.url_no_duplicates</code> (generated column) + unique index | <p>Collapses trivial URL variants (scheme, trailing slash, #fragment — query strings deliberately preserved) so re-inserting a known source is a safe <code>ON CONFLICT DO NOTHING</code> no-op instead of accumulating duplicates. A one-time <code>DO \$\$ ... \$\$</code> cleanup block collapsed real duplicate rows found in production before the unique index was added, repointing <code>research_document_sources</code> links so no provenance was lost.</p>             |
| <code>pipeline_jobs</code> (new table)                                            | <p>One current-state row per (role, company, job_type) unit for the production orchestrator.</p> <p><code>job_type CHECK IN ('ROLE_RESEARCH', 'COMPANY_RESEARCH', 'ROLE_GENERATION');</code> status <code>CHECK</code> across the 13-state lifecycle (PENDING→RESEARCHING→RESEARCHED→SYNTHESIZING→KNOWLEDGE_UPDATED→GENERATING or SKIPPED_NO_CHANGE/WAITING_FOR_QUOTA/BLOCKED/FAILED). Unique on <code>(role, COALESCE(company, ''), job_type)</code> — always safe to upsert.</p> |

## 12. Company Knowledge → RAG Wiring (Deep Dive)

This is the most recently added capability. Before it existed, `execute_company_research()` produced real company research but that knowledge was never actually consumed by question generation — a documented gap. The fix wires company knowledge into the exact same evidence-driven retrieval mechanism role knowledge already used, **without** a schema change and **without** ever injecting the full 24-company list into a prompt.

### 12.1 The storage trick: reusing `role` for company-only chunks

`KnowledgeChunk.role: str` (Pydantic) is required, and `research_chunks.role TEXT NOT NULL` (migration 0001) forbids a null role at the DB level too. Rather than a schema migration, a standalone company chunk stores the **company name in the `role` slot** (exactly the convention `knowledge_state/research_documents` already used for company-only research), while `company` (nullable everywhere) is the actual discriminator every retrieval path checks explicitly. `rag_service.retrieve_context()` filters on `c.company is None` in *both* its primary match and its fallback branch, as defense-in-depth — a company chunk can never leak into role-only retrieval regardless of what ended up in its `role` field.

### 12.2 How company knowledge reaches RAG

```
process_company_research(company)
-> research_service.execute_company_research(company)           [real Tavily+Firecrawl call]
-> synthesize_and_index_company_research(company, output)
    -> synthesis_service.synthesize(output)                     [chunk.role = company, from ResearchOutput.role]
    -> stamp chunk.company = company explicitly
    -> rag_service.index_chunks(chunks)                         [embeds + persists to vector_store.json]
-> db_loader.load_company_research(company)                    [persists to research_documents/research_chunks, DB-]
```

### 12.3 How relevance is decided (never a fixed list)

`rag_service.retrieve_company_context(role, experience_band, paradigm, companies=None, top_k=1):` builds a candidate pool of every indexed chunk with `company` set (optionally restricted to an explicit `companies` list), embeds a query string, ranks the pool by the exact same `_cosine_rank()` core role retrieval uses, and returns only chunks scoring  $\geq$  `config.COMPANY_CONTEXT_MIN_SIMILARITY` (0.35). A company whose knowledge doesn't actually relate to the role/band/paradigm being generated simply never clears the bar — this is what keeps the mechanism evidence-driven *structurally*, not by hardcoding an allow-list. `top_k=1` keeps it strictly supplementary. `generator_service.generate_batch_for_band()` merges any hit into the prompt as an explicitly optional `[Company Context -- <company>, topic: <topic>]` block, with an accompanying rule telling the LLM it is not an instruction to write a company-specific question — a UNIVERSAL/DOMAIN-targeted question with company context present should still end up UNIVERSAL/DOMAIN unless the question's actual content genuinely requires that company.

### 12.4 Company research state machine (`orchestrator.process_company_research`)

```
is_due(company, company)? [freshness.py, 14-day cadence]
no -> pipeline_jobs: SKIPPED_NO_CHANGE ("within cadence window") -- nothing touched
yes -> pipeline_jobs: RESEARCHING
    -> execute_company_research(company)
    -> compare new source_hash to knowledge_state.source_hash
        unchanged -> record_no_change(); pipeline_jobs: SKIPPED_NO_CHANGE -- no synthesis, no RAG index, no
        changed   -> bump_version(); record_change() [+ append knowledge_version_history row]
            -> pipeline_jobs: KNOWLEDGE_UPDATED
            -> synthesize_and_index_company_research()
            -> db_loader.load_company_research()
            -> pipeline_jobs: COMPLETED, metrics={chunks_inserted: N}
```

Scope classification is completely unaffected by any of this: `pipeline_runner.classify_scope()` only ever looks at a candidate's own `applicable_companies/scope` as produced by the LLM — company context reaching the

prompt as evidence never forces COMPANY or DOMAIN scope by itself.

## 13. End-to-End Data Flow Walkthroughs

### 13.1 Legacy CLI run for one role (first run, no prior state)

```
pipeline_runner.run_role_pipeline("Machine Learning Engineer", target_questions=20)
-> validate_prerequisites() [credential check, real mode only]
-> research_service.execute_research(role) [-> Tavily+Firecrawl or Perplexity -> ResearchOutput]
-> synthesis_service.synthesize(research_output) [-> 4-6 KnowledgeChunks via LLM]
-> rag_service.index_chunks(chunks) [-> embedded + saved to vector_store.json]
for band in ["0-1", "1-2", "3-5", "5-8", "8+"]:
    target_scope = UNIVERSAL / DOMAIN (band 2) / COMPANY (band 5)
    generator_service.generate_batch_for_band(role, band, count, target_scope)
        -> rag_service.retrieve_context() + retrieve_company_context()
        -> LLM generate_json() -> List[QuestionObject] candidates
    for each candidate:
        candidate.scope = pipeline_runner.classify_scope(candidate, target_scope)
        gate_result = validator_service.validate_question(candidate, question_bank, expected_role=role)
            -> deterministic checks -> deduplicator_service -> rag_service.retrieve_context()
            -> Gemini (Tier1) -> [OpenAI (Tier2)] -> [Claude (Tier3)] if uncertain
        if approved: append to question_bank.json; else: log rejection reason
        validation_store.record(attempt) [durable per-candidate checkpoint]
        state_manager.update_role_state(current_batch=..., accepted_questions_count=...)
final status = COMPLETED (>=70% of target) or PARTIAL
```

### 13.2 Production orchestrator run for one role (refresh with changed knowledge)

```
RolloutOrchestrator.process_role("Machine Learning Engineer")
freshness.is_due(role)? [DB knowledge_state, 30-day cadence]
-> yes: pipeline_jobs: RESEARCHING
    research_service.execute_research(role, force=True)
-> pipeline_jobs: RESEARCHED
compare new source_hash vs knowledge_state.source_hash
-> changed: pipeline_jobs: SYNTHESIZING
    synthesis_service.synthesize() + rag_service.index_chunks()
    bump_version(); freshness.record_change() [+ knowledge_version_history row]
    pipeline_jobs: KNOWLEDGE_UPDATED
    target_count = REFRESH_TOP_UP_QUESTIONS_PER_ROLE (8) [full pilot size only on first-ever run]
    pipeline_jobs: GENERATING
    existing_bank = fetch_existing_bank_from_db(role) [DB-authoritative dedup context]
    for band in EXPERIENCE_BANDS:
        candidates = generator_service.generate_batch_for_band(...)
        pipeline_jobs: VALIDATING
        for cand: classify_scope() -> validator_service.validate_question(cand, existing_bank+approved)
    pipeline_jobs: LOADING
    db_loader.load_role(role, approved, attempts) [idempotent insert of only the newly-approved]
    pipeline_jobs: COMPLETED, metrics={generated, approved, rejected, inserted, is_first_time}
```

### 13.3 Quality gate walkthrough for one candidate

```
validate_question(candidate, existing_bank, expected_role)
1. deterministic checks (role, difficulty band, paradigm, scope, dimension consistency)
2. deduplicator_service.check_duplicate() [exact -> normalized -> semantic]
3. rag_service.retrieve_context() -- must return >=1 chunk or "insufficient_rag_grounding"
4. Tier 1 (Gemini) LLM rubric, 12 dimensions -> score/passed/uncertain/critical_failures
    - deterministic failure present? -> return rejected immediately (no LLM escalation spent)
    - Tier 1 confident? -> return its result
5. Tier 1 uncertain -> Tier 2 (OpenAI fallback)
    - Tier 2 confident? -> return its result
6. Tier 2 uncertain -> Tier 3 (Claude escalation)
    - Tier 3 still uncertain? -> FORCE REJECT (unresolved_uncertainty_after_full_ladder)
7. Tier 2/3 unreachable (no credentials / quota) -> FORCE REJECT
    (escalation_unavailable_uncertain_result) -- never fall back to Tier 1's shaky score
RESULT: approved = (not critical_failure) and passed and not uncertain -- ALWAYS, no exceptions
```

## 14. Testing Strategy

133 tests total across 9 files, all run against `MOCK_MODE=true` (zero real API cost) except where a test file explicitly documents exercising the real (migrated) Postgres schema with obviously-synthetic role/company names (prefixed `___Test.___`), always purged in `setUp/tearDown`. Test isolation for every local file the pipeline touches is enforced by `tests/__init__.py`, which sets `QUESTION_PIPELINE_DATA_DIR` to a fresh temp directory *before* any pipeline module is imported — this must be run via the package path (`python -m unittest question_pipeline.tests.test_x ...`); a bare `unittest discover` without an explicit top-level directory can bypass this bootstrap and must never be used against this project.

| File                                            | Focus                                                                                                                                                                                                                                                                                                                      |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>test_pipeline.py</code>                   | Core pipeline mechanics: dedup tiers, RAG indexing/retrieval, embedding-model invalidation.                                                                                                                                                                                                                                |
| <code>test_quality_gate.py</code>               | Fail-closed enforcement: a critical failure can never be approved regardless of score; escalation ladder behavior.                                                                                                                                                                                                         |
| <code>test_data_isolation.py</code>             | Proves real <code>question_pipeline/data/</code> is never touched by a correctly-invoked test run.                                                                                                                                                                                                                         |
| <code>test_tavily_firecrawl_provider.py</code>  | The real research provider's discovery/extraction/fallback/failure paths.                                                                                                                                                                                                                                                  |
| <code>test_scope_and_generation_fixes.py</code> | Regression coverage for the scope-classification bug fix and the copied-example-company-list bug fix.                                                                                                                                                                                                                      |
| <code>test_fallback_and_json_recovery.py</code> | Malformed-JSON recovery ( <code>extract_json_object</code> , bounded retry) and validation-tier fallback behavior.                                                                                                                                                                                                         |
| <code>test_db_loader.py</code>                  | Real-DB integration tests for <code>QuestionBankLoader</code> : idempotency, provenance linkage, transaction rollback, legacy-table isolation.                                                                                                                                                                             |
| <code>test_orchestrator.py</code>               | Real-DB integration tests for the production orchestrator: config validation, freshness cadence, job checkpointing, first-run/no-change/incremental-refresh/quota/blocked/dry-run scenarios.                                                                                                                               |
| <code>test_company_rag_context.py</code>        | The company-RAG feature: role-only retrieval unaffected, relevant/irrelevant company context retrieval, company evidence producing COMPANY/DOMAIN scope only when genuinely justified, company research never forcing scope, company research cadence/change-detection, provenance and dedup with company-chunk citations. |

## 15. Operational Reference

### 15.1 Legacy CLI

```
python -m question_pipeline.pipeline_runner --role "Data Engineer" --count 20
python -m question_pipeline.pipeline_runner --pilot
python -m question_pipeline.pipeline_runner --role "Data Engineer" --force
python -m question_pipeline.pipeline_runner --status
```

### 15.2 Production rollout CLI

```
python -m question_pipeline.rollout_cli --dry-run --pilot
python -m question_pipeline.rollout_cli --pilot --batch-size 2
```

```
python -m question_pipeline.rollout_cli --roles "Data Scientist,Prompt Engineer" --batch-size 2
python -m question_pipeline.rollout_cli --companies "Stripe,Netflix" --batch-size 2
python -m question_pipeline.rollout_cli --pilot --companies "Netflix" --batch-size 1
python -m question_pipeline.rollout_cli --all-roles --batch-size 3          # requires explicit --all-roles
python -m question_pipeline.rollout_cli --status
```

### 15.3 Key environment variables

| Variable                                                                                                                                                                                    | Purpose                                                                             |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <code>MOCK_MODE</code>                                                                                                                                                                      | true = every provider factory returns a mock/local implementation, zero real calls. |
| <code>RESEARCH_PROVIDER</code> / <code>LLM_PROVIDER</code> / <code>EMBEDDING_PROVIDER</code>                                                                                                | Selects the real backend for each capability.                                       |
| <code>FALLBACK_LLM_PROVIDER</code> / <code>ESCALATION_PROVIDER</code>                                                                                                                       | Tier 2 / Tier 3 of the validation ladder.                                           |
| <code>GEMINI_API_KEY</code> , <code>OPENAI_API_KEY</code> , <code>ANTHROPIC_API_KEY</code> , <code>TAVILY_API_KEY</code> , <code>FIRECRAWL_API_KEY</code> , <code>PERPLEXITY_API_KEY</code> | Provider credentials.                                                               |
| <code>QUESTION_PIPELINE_DATA_DIR</code>                                                                                                                                                     | Test-only redirect of every local data path; never set in production.               |
| <code>COMPANY_CONTEXT_MIN_SIMILARITY</code>                                                                                                                                                 | Cosine-similarity floor for company context to surface (default 0.35).              |
| <code>REFRESH_TOP_UP_QUESTIONS_PER_ROLE</code>                                                                                                                                              | Batch size for an incremental refresh vs. a first-ever run (default 8).             |

### 15.4 Cadences

Role research: 30 days. Company research: 14 days. Both are DB-authoritative for the production orchestrator (`knowledge_state.next_research_at`) and file-authoritative for the legacy CLI (`state.json`).

## 16. Key Invariants & Design Rules (Glossary)

**Fail-closed, always.** A high LLM score can never override a deterministic or LLM-flagged critical failure. Enforced in `ValidatorService._merge()` and, once migration 0001 is applied, redundantly at the database level via `chk_qva_fail_closed`.

**Role invariance.** Experience level changes difficulty/depth, never the role. The generator force-overwrites `role` on every candidate regardless of what the LLM returned, and the validator's first deterministic check rejects any mismatch.

**One question, many companies.** `questions` holds one canonical row per question; `question_companies/question_domains/question_skills` are many-to-many joins, never physical duplication per company.

**Scope is content-driven, never assumed.** UNIVERSAL/DOMAIN/COMPANY is decided by `classify_scope()` from the candidate's own genuine `applicable_companies` content, not from generation intent or from the mere presence of company context in the RAG prompt.

**DB is authoritative for the production path.** The production orchestrator always re-derives dedup context and freshness state from Postgres, never from local files or in-process memory alone.

**Never fabricate research.** Both the Perplexity and Tavily+Firecrawl providers raise rather than substitute placeholder content when a genuine source can't be found or extracted.

**Provenance is always traceable.** Every approved question's `source_references` can be resolved back to a real `research_chunks` row (role or company) via `question_sources`, or, failing that, an honest raw citation string.

**Idempotent by construction.** Every insert path (`questions`, `research_chunks`, `research_sources`, `question_embeddings`, `pipeline_jobs`) relies on a real unique constraint plus `ON CONFLICT`, never an application-level "check then insert" race.

**Company context is evidence, never an instruction.** Retrieval is threshold-gated cosine similarity against real indexed company knowledge; an irrelevant company simply never surfaces, and its mere presence in a prompt never forces COMPANY/DOMAIN scope.